

实验三：跨平台应用开发

一、实验基本信息

实验编号：d20301035103、d20301009203

学时分配：4 学时

实验类型：验证型

每组人数：6 人

对应课程目标：课程目标 2

二、实验目的

1. 掌握Flutter 跨平台应用开发技术
2. 掌握React Native 跨平台应用开发技术
3. 掌握Uniapp 跨平台应用开发技术
4. 掌握MAUI 跨平台应用开发技术
5. 理解不同跨平台框架的技术特点
6. 学会根据项目需求选择合适的跨平台框架

三、实验环境

3.1 Flutter 开发环境

开发工具：Android Studio / VS Code

开发语言：Dart

UI 框架：Material Design

状态管理：Provider

网络请求：http 包

下拉刷新：pull_to_refresh 包

3.2 React Native 开发环境

开发工具：VS Code

开发语言：JavaScript/TypeScript

UI 框架：React Native Components

网络请求：Axios

下拉刷新：RefreshControl

3.3 Uniapp 开发环境

开发工具：HBuilderX

开发语言：Vue.js

UI 框架：Uniapp Components

状态管理：Pinia（可选）

网络请求：uni.request

3.4 MAUI 开发环境

开发工具：Visual Studio 2022

开发语言：C#

UI 框架：XAML / MAUI Components

状态管理：CommunityToolkit.Mvvm

网络请求：HttpClient

四、实验内容

4.1 核心任务

开发业务数据列表页，实现以下功能：

- RESTful API 网络数据请求
- JSON 数据解析
- 下拉刷新功能

- 错误处理与加载状态显示

4.2 实战案例

开发"智慧校园"新闻列表模块，从 RESTful API 获取新闻数据，解析 JSON 并展示列表，实现下拉刷新功能。

数据模型定义

```
// Flutter 数据模型
class News {
  final int id;
  final String title;
  final String content;
  final String author;
  final String publishDate;
  final String imageUrl;

  News({
    required this.id,
    required this.title,
    required this.content,
    required this.author,
    required this.publishDate,
    required this.imageUrl,
  });

  factory News.fromJson(Map json) {
    return News(
      id: json['id'],
      title: json['title'],
      content: json['content'],
      author: json['author'],
      publishDate: json['publishDate'],
      imageUrl: json['imageUrl'],
    );
  }
}
```

五、实验步骤

5.1 Uniapp 列表开发

步骤 1：创建 Uniapp 项目

- 1. 打开 HBuilderX
- 2. 新建 uni-app 项目
- 3. 配置 pages.json 路由

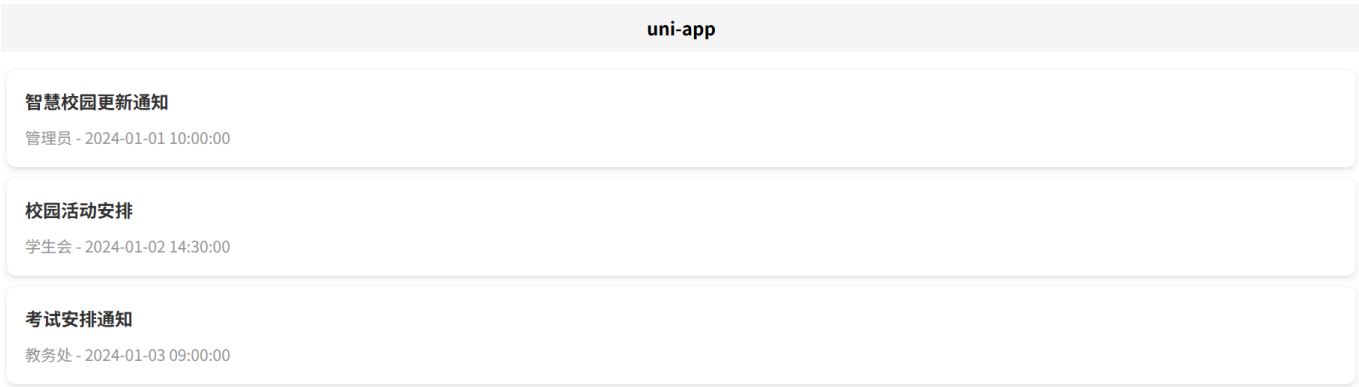
步骤 2：实现新闻列表页面

- 1. 在 data 中定义 newsList 数组
- 2. 在 onLoad 中调用 fetchNews 方法
- 3. 使用 v-for 渲染列表
- 4. 使用 scroll-view 实现下拉刷新

六、实验结果

6.1 功能实现截图

6.1.3 Uniapp 新闻列表界面



Uniapp 界面特点：

- 微信小程序风格设计
- 圆角卡片布局
- 统一的视觉风格
- 多端适配能力强
- 简洁明了的排版

七、实验总结

7.1 技术收获

通过本次实验，我们掌握了四种主流跨平台移动开发框架的基本使用方法：

1. Flutter：学会了使用 Dart 语言和声明式 UI 开发方式，掌握了 Provider 状态管理模式
2. React Native：学会了使用 JavaScript/TypeScript 和 React 组件化开发方式
3. Uniapp：学会了使用 Vue.js 和 uni-app 特有的跨平台开发方式
4. MAUI：学会了使用 C# 和 XAML/MVVM 开发方式

7.2 项目结构

```
OLAPDYnjj/  
├── flutter_news_list/    # Flutter 项目  
│   ├── lib/  
│   │   ├── main.dart    # 主程序文件  
│   │   └── pubspec.yaml  # 项目配置  
├── react_native_news_list/ # React Native 项目  
│   ├── App.js           # 主程序文件  
│   └── package.json      # 项目配置  
├── uniapp_news_list/     # Uniapp 项目  
│   ├── pages/  
│   │   ├── index/index.vue # 列表页面  
│   │   └── detail/detail.vue # 详情页面  
│   ├── pages.json        # 路由配置  
│   └── manifest.json     # 应用配置  
└── maui_news_list/       # MAUI 项目  
    ├── MauiNewsList.csproj # 项目文件  
    ├── ViewModels/  
    │   └── MainViewModel.cs # 视图模型  
    ├── Models/  
    │   └── NewsItem.cs      # 数据模型  
    ├── MainPage.xaml       # 主页面  
    └── App.xaml.cs         # 应用入口
```

7.3 框架选择建议

根据不同的项目需求，建议如下：

- 追求性能和体验：选择 Flutter
- 快速开发和迭代：选择 React Native 或 Uniapp
- 多端统一开发：选择 Uniapp
- .NET 技术栈：选择 MAUI

7.4 后续学习建议

1. 深入学习各框架的高级特性
2. 学习状态管理方案（Redux、Vuex、Riverpod 等）
3. 学习路由和导航管理
4. 学习性能优化技巧
5. 学习自动化测试和持续集成

八、思考与讨论

8.1 跨平台框架的优缺点

优点：

- 降低开发成本，一套代码多端运行
- 提高开发效率，加快产品迭代
- 降低维护成本，统一代码库

缺点：

- 性能可能不如原生应用
- 平台特定功能可能受限
- 学习成本和调试复杂度增加

8.2 如何选择合适的跨平台框架

1. 明确项目需求和目标平台
2. 评估团队技术栈和经验
3. 考虑项目周期和预算
4. 评估框架的成熟度和社区支持
5. 进行技术选型验证和原型开发

8.3 跨平台开发的发展趋势

1. 框架性能持续优化
2. 原生能力支持不断增强
3. 开发体验和工具链完善
4. 跨平台与原生开发的融合